FedDOT: Defending Federated Learning Against Overwhelming Targeted Attacks

Priyesh Ranjan, *Student Member, IEEE*, Ashish Gupta, *Member, IEEE*, Federico Corò, *Member, IEEE*, and Sajal K. Das, *Fellow, IEEE*

***Abstract*—Federated learning (FL), which facilitates collaborative model training and protects users' privacy, has drawn great interest from the research community. With FL, participants train their models on local data and submit the corresponding updates for aggregation to a server. While concealing the identities of the participants, FL may attract adversaries in order to hamper the underlying model. In this article, we propose an FL framework, FedDOT, to defend against adversaries performing *targeted attacks*. FedDOT incorporates two powerful defense algorithms, maximum spanning tree-based attacker detection (MST-AD) and densest graph-based attacker detection (density-AD), which leverage correlation between weight updates and graph theory concepts, maximum spanning tree, and densest graph. With a goal to withstand an overwhelming number of attackers, our algorithms provide strong solutions to aid an FL server, even in overwhelming scenarios where adversaries constitute more than half of the participants. Along with theoretical bounds in correlation space, a rigorous experimental analysis using image classification datasets is carried out to validate the robustness of the FedDOT framework in non-IID settings, which ascertains the superiority of the models against the state-of-the-art methods using a variety of metrics evaluating the accuracy and attack detection rate. With an attack success rate of $< 10\%$ for targeted attacks like single-label flipping, multilabel flipping, and backdoor, FedDOT successfully defends against overwhelming adversaries with a marginal accuracy drop of less than 2%.**

***Impact Statement*—Federated learning (FL) enables collaborative model training while preserving data privacy, but remains vulnerable to adversarial manipulation. This article proposes FedDOT, a robust defense framework designed to enhance the security of FL against targeted attacks. FedDOT incorporates two graph-theoretic detection algorithms—maximum spanning tree-based attacker detection (MST-AD) and densest graph-based attacker detection (density-AD)—that exploit correlations among model updates to effectively identify and mitigate malicious participants. Comprehensive experiments on image classification tasks under non-IID conditions demonstrate that FedDOT achieves strong resilience against single-label flipping, multilabel flipping, and backdoor attacks, maintaining attack success rates below 10% with less than 2% degradation in model accuracy. Theoretical analysis further verifies its stability even when adversarial participants constitute a majority. FedDOT thus represents a significant advancement toward secure and trustworthy FL, enabling reliable deployment of privacy-preserving AI systems in sensitive and safety-critical environments.**

***Index Terms*—Attack detection, backdoor, federated learning (FL), targeted attackers.**

Nomenclature

T	Training rounds.
ρ	Correlation between client weight updates.
$\mathcal{A}$	Graph adjacency matrix on weight correlations.
$\mathcal{G}$	Correlation graph.
$\mathcal{T}$	Spanning tree on $\mathcal{G}$.
$\mathcal{V}$	List of sparse vertices in $\mathcal{G}$.
δ_i	Local model update of client i.
W^t	Global model weights at round t.

I. Introduction

With the prevalence of the Internet of Things (IoT), there is an abundance of data-collecting devices capable of performing lightweight computation. Through collaborative learning, these devices can produce robust machine learning models by sharing data with a central server, which, however, raises privacy concerns for the participants. Federated learning (FL) [1] has emerged as a new paradigm for privacy-preserving collaborative learning, in which each participating device trains a local model using local data and sends weight updates to the server that aggregates the updates to get a global model, which is distributed back to the participants. These steps are repeated multiple times (called FL rounds) to train an optimal model. Recently, FL has gained traction in various fields such as IoT [2], healthcare [3], and natural language processing [4], [5].

Due to the server's lack of control over participants, FL has been targeted by various adversarial attacks from participants (clients) with malicious intent. According to the literature, two well-studied forms of attack include the injection of corruption into local data before training [6], [7], [8], [9] and the modification of model updates after training [10], [11]. The former

Received 17 October 2025; revised 4 February 2026; accepted 16 March 2026. Date of publication 23 March 2026; date of current version 31 August 2026. The work of P. Ranjan and S. K. Das was supported by the U.S. National Science Foundation (NSF) under Grant ECCS-2319995, Grant OAC-2104078, Grant SaTC-2030624, and Grant CNS-208878. This article was recommended for publication by Associate Editor Jingfeng Zhang upon evaluation of the reviewers' comments. *(Corresponding author: Priyesh Ranjan.)*

Priyesh Ranjan and Sajal K. Das are with the Department of Computer Science Missouri University of Science and Technology, Rolla, MO 65409 USA (e-mail: pr8pf@mst.edu; sdas@mst.edu).

Ashish Gupta is with the Department of Computer Science and Engineering BITS Pilani, Dubai 345055, UAE (e-mail: ashish@dubai.bits-pilani.ac.in).

Federico Corò is with the Department of Mathematics University of Padua, 35122 Padua, Italy (e-mail: federico.coro@unipd.it).

Digital Object Identifier 10.1109/TAI.2026.3676747

refers to *targeted attack*, in which multiple adversarial clients attempt to corrupt a targeted class while keeping the overall performance of the model almost intact, making their detection difficult. Some common targeted attacks include flipping the labels of the local data shard [12], [13] and embedding a particular trigger pattern into instances of a particular class [6], [7], [8]. The second form is *untargeted attack*, in which the adversaries aim to corrupt the whole model via additive noise or sign-flipping [9], [14].

1) *Motivation*: While existing defenses can mitigate the attacks, they suffer from the following limitations.
   a) Most prior solutions, e.g., [1], [12], [15], [16] rely on the assumption that the number of adversaries is less than the number of benign participants, thus limiting their application to severe FL scenarios where this assumption does not hold.
   b) In the presence of overwhelming attackers, the widely adopted distance metrics, such as Euclidean and Cosine similarity, fail to distinguish adversarial models from benign ones.
   c) The assumptions about client weight updates, to leverage clustering algorithms like in [17], fall short in practical settings with nonindependent and identical distribution (non-IID) data, inherent to FL settings.

This article addresses the above limitations by introducing a novel FL framework, abbreviated as *FedDOT*, that can defend against overwhelming targeted attacks (i.e., the number of adversaries is more than half of the total number of participants) in non-IID settings.

1) *Major Contributions:* The major contributions of this work are as follows.
   a) We propose a novel FedDOT framework incorporating two graph theoretic algorithms, named maximum spanning tree-based attacker detection (MST-AD) and densest graph-based attacker detection (density-AD), to identify adversaries and mitigate their impact on the global model.
   b) By strategically utilizing the correlation between the local models (i.e., weight updates), both MST-AD and density-AD ascertain that the targeted adversaries are closer to each other than the normal clients, and leverage this information to filter them out from the aggregation.
   c) FedDOT provides theoretical bounds in the correlation space and establishes necessary and sufficient conditions for the detection of all adversaries.
   d) We carry out an extensive set of experiments to validate the resilience of our framework against diverse targeted attacks using three benchmark datasets. We especially simulate scenarios with overwhelming adversaries colluding against benign participants to evaluate the effectiveness of MST-AD and density-AD using various performance metrics and show the superiority of FedDOT over existing solutions.

This article significantly extends our preliminary work [18], [19] on the following aspects.

1) While the previous versions of our algorithms (MST-AD and density-AD) could only detect *single-label flipping* and a few cases of *backdoor attacks*, the FedDOT framework allows the algorithms to handle *multilabel flipping* and a larger variety of *backdoor attacks*.
2) FedDOT can isolate adversaries even when they outnumber benign participants. This difficult challenge, once solved, creates new possibilities for future research into severely attacked scenarios.
3) In our prior research, we relied on experimental evaluation to prove efficacy. In addition to that, FedDOT offers strong theoretical guarantees (including convergence analysis) for identifying and isolating adversaries for both algorithms.
4) We also include an experimental analysis using an additional dataset and compare our findings to newer state-of-the-art algorithms that have been introduced recently.

The remainder of the article is organized as follows. Section II discusses related work. Section III describes our FL framework with the considered threat model. Section IV proposes the attacker detection algorithms, and Section V evaluates them on two benchmark datasets. Finally, we conclude the article in Section VI.

II. Related Work

While existing works can defend against adversaries in FL, their effectiveness is bounded by assumptions on the attacker's capability and intensity. We categorize them based on the objectives and underlying approaches.

A. Defense Against Untargeted Attacks

Untargeted attacks aim to disrupt the global model's convergence by submitting weight updates that hinder its performance. Existing defenses include techniques like local training validation [20], bootstrapping to defend against adaptive attacks [21], stochastic aggregation with regularization [22], and trigger-based methods [23]. The work [24] leverages client-side masking of the weight updates followed by aggregation over the masked deltas, leading to client-side privacy and robust aggregation. We excluded these attacks from our study because they are relatively easy to detect by analyzing update convergence.

B. Aggregation Using Central Measures

Many defenses against adversaries in FL rely on central statistics, assuming that the attackers are in the minority. Examples include mean and trimmed mean-based gradient descent [16], median-based batch gradient descent [25], and distributed stochastic averaging gradient algorithm [26]. Dynamic learning rates have also been explored to maximize the loss for the backdoor attackers [27], but this approach is effective only when the number of participating clients is below a certain threshold. The assumption of adversaries being less than a dynamic threshold is presented in Li et al. [28], again leading to ineffective defense against overwhelming adversaries. The works in Ranjan et al. [29], [30] also aim to mitigate distributed backdoor attacks by isolating weight updates with increased

cosine distances, but fail in scenarios where the number of adversaries exceeds the number of benign clients. Recently, Nguyen et al. [31] has devised sequential strategies to clip and filter adversarial participants using cosine distances. However, they assume the adversarial participants to be in the minority, which limits the impact of the defense. It is important to note that complete mitigation of adversaries is not possible through central tendency measures, especially in the presence of overwhelming adversaries, as they can pull centrality towards them.

C. Isolation of Suspected Adversaries

Some existing algorithms achieve robustness by isolating suspected clients. However, their reliance on unrealistic assumptions limits their effectiveness against overwhelming adversaries. For example, Krum [15] is a popular aggregation algorithm that excludes distant weight update vectors to achieve resilience. However, in the presence of an overwhelming number of attackers, the notion of far-laying vectors may lead to the elimination of the complementary set consisting of benign clients. To overcome this limitation, FoolsGold [12], used as a benchmark in this work, assumes a higher similarity between adversarial weight updates. However, as established by our results, it requires multiple rounds for detection and, at times, results in benign participants being flagged as adversaries, especially under label-flipping attacks. A similar approach used in Ranjan et al. [32] leads to limitations for the defense. In contrast, the defense algorithm in Mao et al. [33] utilized divergence between participants to detect instances of model poisoning. However, its tolerance of adversaries, rather than their isolation, reduces its efficacy.

D. Update History-Based Defenses

Recently, Gupta et al. [14] used the history of gradients obtained over multiple rounds to identify the set of targeted and untargeted attackers. However, the proposed method is ineffective against targeted backdoor attacks. Similarly to FoolsGold, in Awan et al. [34], pairwise similarity between aggregated historical updates is utilized along with dynamic learning rates and the probability of selection of each client for the next communication round. However, the alignment rate of each client is calculated using top-k pairwise cosine similarities, requiring a predetermined attacker population, which is unrealistic.

E. Latent Space Representation-Based Defenses

In [35], the authors propose a defense mechanism that leverages the penultimate layer representations of the model to evaluate the trustworthiness of each client's update, assigning lower scores to those far from a benign cluster. This method is designed to be resistant to various model poisoning attacks, including semantic backdoor, trojan backdoor, and untargeted attacks. However, we remark that our framework differs by using graph theory and analyzing the correlation between weight updates to detect attackers. This allows FedDOT to be effective even when the number of adversaries is overwhelming and the data is non-IID, conditions where many other methods, including those that might rely on an "honest majority" assumption, may struggle.

III. Problem Description

This section provides the background of FL and the objectives of the FedDOT framework. It also states the threat posed by the adversaries with their capabilities.

A. Federated Learning

A standard FL framework includes a central server S and a set of client devices denoted by $\mathcal{C}$. Let $\mathcal{M} \subset \mathcal{C}$ denote the subset of adversarial clients and $\mathcal{B} = \mathcal{C} \setminus \mathcal{M}$ the corresponding set of benign workers. Each client $c_i \in \mathcal{C}$ holds its own data shard $\mathcal{D}_i = \{\mathbf{X_i}, \mathbf{y_i}\}$ where $\mathbf{X_i}$ is the set of training samples and $\mathbf{y_i}$ is the corresponding set of labels. We assume that the server is unbiased and is not inclined to intrude into the clients' data shards. The server S distributes the global model W^t to clients at the start of the communication round t. Each participant c_i computes the corresponding gradient vector on their local data shard as

$\delta_i^t = W^t - \eta \cdot l_i(W^t, \mathcal{D}_i)$, where η and l_i are the learning rate and the loss function at client c_i, respectively. The gradient vector (weight update) is then communicated to the server that computes a global model using FedAvg [1] for round $t+1$ as follows:

$$W^{t+1} = W^t + \sum_{c_i \in \mathcal{C}} p_i \delta_i^t, \quad \text{where} \quad p_i = \frac{|\mathcal{D}_i|}{\sum_{c_i \in \mathcal{C}} |\mathcal{D}_i|}. \tag{1}$$

Later, the updated model W^{t+1} is distributed to all clients. This process is repeated until the collaborative model approaches the converged model W^*.

B. Aggregation Objective

FedDOT aims to identify the set of adversary-controlled clients $\mathcal{M}$ and manually enforce the contribution of their updates to zero during aggregation. Qualitatively, we enforce $\delta_j = 0, \forall\ c_j \in \mathcal{M}$ in (1). Thus, the aggregation in FedDOT is done as follows:

$$W^{t+1} = W^t + \frac{1}{\sum\limits_{c_i \in \mathcal{B}} |\mathcal{D}_i|} \sum_{c_i \in \mathcal{B}} |\mathcal{D}_i| \delta_i^t \tag{2}$$

which replaces line 3 in Algorithm 1 for the aggregation.

C. Threat Model

We consider targeted attacks done by a single adversary having control over a large number of clients. The adversary injects poison into their respective local data shards and trains the models using that data. In particular, this work considers two types of attack.

1) *Label Flipping Attack:* It involves swapping of some labels with others in $\mathcal{D}_i$. For example, the adversary flips a single label "A" to "B" and vice versa, as mentioned in [19]. In FedDOT, we also simulate scenarios with the

Algorithm 1: FedAvg.

Input: Initial weights W^0
Output: Optimal weights W^*
At Server:
for $t = 1$ *to* T **do** // T: Total number of rounds
  for *each client* $c_i \in \mathcal{C}$ **do**
    $\delta_i^t \leftarrow \texttt{Client}(W^t)$
  $W^{t+1} \leftarrow W^t + \frac{1}{\sum_{c_i \in \mathcal{C}} |\mathcal{D}_i|} \sum_{c_i \in \mathcal{C}} |\mathcal{D}_i| \delta_i^t$
return $W^* = W^{t+1}$

function Client(W^t):
  $B \leftarrow$ batches$(\mathcal{D}_i)$ // B: Number of batches
  for $e \in E$ **do** // E: Number of epochs
    for *batch* $b \in B$ **do**
      $\delta^t \leftarrow W^t - \eta \cdot l(W^t, b)$
  return δ^t

adversary flipping multiple labels at once in their data shards (say "A," "B" to "C," "D," respectively).

2) *Backdoor Attack:* The adversary adds a trigger pattern into some selected data samples/images (of $\mathcal{D}_i$) and flips their labels with a target label. This makes the adversarial model responsive to the embedded pattern instead of true classification. We rigorously simulate overwhelming backdoor attacks with divergent trigger locations to ascertain the robustness of the aggregation.

D. Attacker Capabilities and Assumptions

The adversary has full control of compromised devices, but no access to the training data or process of benign clients. Moreover, the adversary has no information about the detection of controlled clients in the previous rounds and can not alter the attacking strategy over the entire training. FedDOT assumes that the number of benign clients is at least two to separate them from adversaries.

Further, we relax the assumption that the client data shards have to follow the IID distribution. While such an assumption is frequently made by existing works [25], [36], it invalidates the heterogeneous nature of the clients.

Some of the frequently used notations in our work are listed in the nomenclature.

IV. FedDOT Framework

We introduce our FedDOT framework and present two novel graph-theoretic algorithms, MST-AD and density-AD. These algorithms have been specifically designed to detect targeted adversaries within our framework. Later, we discuss the theoretical bounds concerning the detection of adversaries.

A. Similarity Between Weight Updates

As mentioned in [12], [14], FedDOT assumes that weight updates from adversaries (sharing a common objective) are more similar to each other compared to the similarity among benign clients. Moreover, adversaries exhibit high divergence from benign clients. We generalize this assumption to non-IID settings for the attack types described in Section III-C.

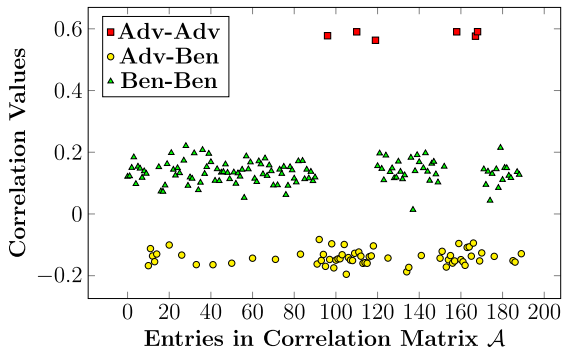

Fig. 1. Illustrating correlation values between different clients. The setup contains a total of 20 clients, out of which four are controlled by the adversary performing a targeted attack.

This similarity between clients is quantified using the cosine similarity of aggregated updates across multiple rounds [12], [14], [34]. However, we only consider the current training round for measuring similarity. This is to avoid adversaries that pose as benign clients for a large part of the training process. Particularly, we employ Pearson's correlation coefficient to quantify this similarity among weight updates.[1] We specifically choose Pearson's correlation because it is invariant to both the scaling and shifting of the weight update vectors, making it more robust to variations in local learning rates or update magnitudes across heterogeneous clients. For any two clients c_i and c_j, the correlation similarity is computed as follows:

$$\rho_{\delta_i,\delta_j} = \frac{Cov(\delta_i, \delta_j)}{\sigma_{\delta_i}\sigma_{\delta_j}}$$

where $Cov(\cdot,\cdot)$ denotes the covariance between the weight updates δ_i and δ_j, while σ represents the standard deviation. For the sake of clarity, we use ρ_{ij} instead of ρ_{δ_i,δ_j} throughout the remainder of this article.

We consider a representative example of an FL framework with 20 clients (including four label-flipping attackers) collaborating on image classification using the MNIST dataset [37], and report the correlation values in Fig. 1. It is easy to observe that the correlation between two adversarial clients (shown in red) is higher than that between two benign clients (shown in green), which in turn is higher than that between an adversarial and a benign client (shown in yellow). Leveraging this observation, FedDOT makes the following assumption: given a set of clients $\mathcal{C}$ and a set of attackers $\mathcal{M} \subset \mathcal{C}$, the inequality

$$\rho_{jp}, \rho_{jq}, \rho_{iq}, \rho_{ip} < \rho_{ij} < \rho_{pq} \tag{3}$$

holds, where the clients $c_p, c_q \in \mathcal{M}, p \neq q$ and the clients $c_i, c_j \in \mathcal{B} = \mathcal{C} \setminus \mathcal{M}, i \neq j$.

While this correlation inequality provides a strong heuristic for separating colluding adversaries, we acknowledge scenarios where it might be less pronounced. For instance, in

[1]We focus on the output-layer weights for similarity computation, as they directly map to the target and are thus more sensitive to adversarial poisoning.

highly heterogeneous non-IID settings, two benign clients with very similar data distributions could exhibit an unusually high benign–benign correlation. In contrast, multiple distinct groups of adversaries targeting different classes might show weaker adversary–adversary correlation than a single, unified attacking group. However, our extensive experimental results suggest that, on average, updates from clients sharing a malicious, targeted objective still produce a statistically significant and separable correlation signal, validating this assumption as a robust foundation for our defense in practice.

B. Graph Representation

To take advantage of graph theory concepts, we transform the correlation matrix (obtained in Section IV-A) into a graph representation. Let $\mathcal{A} \in \mathbb{R}^{n\times n}$ be a correlation matrix, where $n = |\mathcal{C}|$. This matrix is symmetric along its axis as $\rho_{ij} = \rho_{ji}, \forall\, c_j, c_i \in \mathcal{C}$ and the values lie in the range $[-1, 1]$. The diagonal entries ρ_{ii} are made 0, $\forall c_i \in \mathcal{C}$. An example of $\mathcal{A}$ is shown below

$$\mathcal{A} = \begin{array}{c} \\ c_1 \\ c_2 \\ \cdots \\ c_n \end{array} \begin{array}{c} \begin{array}{cccc} c_1 & c_2 & \cdots & c_n \end{array} \\ \begin{pmatrix} 0 & \rho_{12} & \cdots & \rho_{1n} \\ \rho_{21} & 0 & \cdots & \rho_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ \rho_{n1} & \rho_{n2} & \cdots & 0 \end{pmatrix} \end{array}.$$

FedDOT uses $\mathcal{A}$ as a graph adjacency matrix to define a graph $\mathcal{G}$ with n clients as vertices and $\binom{n}{2}$ correlation values as edge weights. Thus $\mathcal{G}$ is a complete weighted graph with edge weights in $[-1, 1]$ and free from self-loops as diagonal entries in $\mathcal{A}$ are zero. As we use $\mathcal{G}$ in the attacker detection algorithm, we use the terms adjacency matrix and correlation matrix interchangeably while referring to $\mathcal{A}$.

Example: To illustrate our approach, let us consider a scenario involving five clients: $c_1, c_2, c_3, c_4, c_5 \in \mathcal{C}$. Fig. 2 shows their respective weight updates $\delta_1, \delta_2, \delta_3, \delta_4, \delta_5$. In this example, clients c_1 and c_4 (highlighted in red) are targeted attackers attempting to poison the global model. By computing correlation values using weight updates, as shown on the edges, we can obtain support for the inequality described in (3). This graph formulation serves as a basis for our detection algorithms in subsequent sections.

Following this, we propose a precursory check based on support-based contamination to minimize the instance of false positive is scenarios with no attackers. We describe the check as given below.

1) We compute the max and min value across the correlation matrix $\mathcal{A}$ as $max = \max(\mathcal{A})$ and $min = \min(\mathcal{A})$
2) We obtain the average values as $\mu = avg(\mathcal{A})$.
3) We compute the upper support $\mathrm{u} = \mu + 0.3r$ where r is the range of the observations $r = max - min$.
4) The soft probability for the existence of adversarial participants is computed as $P(\mathcal{M}) = \sigma(max - \mathrm{u})/(\beta(\mathrm{u} - \mu))$ where β is chosen as 0.4 and σ is the sigmoid function [38].
5) **If** $P(\mathcal{M}) < 0.5$, no defense is required, *Else* execute the proposed defenses mentioned in Sections IV-C and D.

Now we explain the MST-AD and density-AD algorithms performed after the check.

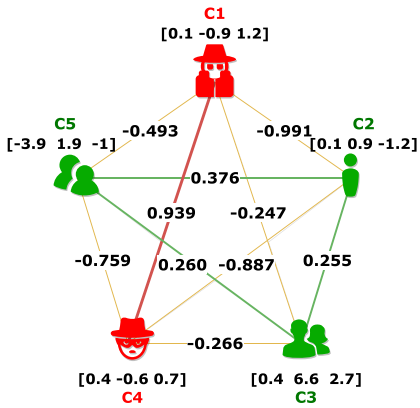

Fig. 2. Example graph $\mathcal{G}$ with five clients and correlation values as edge weights. c_1, and c_4 are targeted attackers. Numeric values with each client represent their weight updates.

C. MST-AD Algorithm

This section proposes our first detection algorithm, MST-AD, by constructing a maximum spanning tree (MST), which is defined below for a quick reference.

Definition 1: An MST $\mathcal{T}$ is a subgraph connecting all n vertices (clients) using $n-1$ edges with maximum weights (correlation values between clients), ensuring no cycles.

1) *Creation of MST*: For a given graph $\mathcal{G}$ with n vertices, we employ Kruskal's algorithm [39] to obtain an MST $\mathcal{T}$ as follows: We start by adding the edge with the highest correlation ($\arg\max_{i,j} \rho_{ij}$) to an empty tree. Then, we iteratively add the next highest weighted edge, ensuring no cycles form, until we have $n-1$ edges in the tree.
2) *Attacker Detection Process*: In the presence of attackers, $\mathcal{T}$ should have two subtrees with a proper separation of adversarial clients from benign ones, assuming that the inequality stated in (3) holds. These two subtrees would be connected via a single adversarial-benign edge with the lowest weight in $\mathcal{T}$. Removal of this single edge would result in two disjoint subtrees $\mathcal{T}_1^{\text{sub}}$ and $\mathcal{T}_2^{\text{sub}}$. If we assume that the number of adversaries is less than the number of benign ones, then the small subtree would correspond to the attackers. However, such an assumption does not hold in FedDOT as it is designed to deal with overwhelming attackers. However, based on (3), the attacker's subtree always has a higher median edge weight, regardless of the percentage of attackers. Algorithm 2 summarizes the process.

Example 1: For a better understanding of the MST-AD algorithm, we provide an example on the representative graph shown in Fig. 3. First, the edge ρ_{14} is added to the set $\mathcal{T}$, followed by ρ_{25} and ρ_{35}, respectively. However, the edge ρ_{23} is not added as it creates a cycle within $\mathcal{T}$. The next edge, ρ_{13}, is added, resulting in the maximum spanning tree over $\mathcal{G}$. Next, the smallest edge, ρ_{13}, is removed from $\mathcal{T}$ to form two subtrees, $\mathcal{T}_1^{\text{sub}} = c_1, c_4$ and $\mathcal{T}_2^{\text{sub}} = c_2, c_3, c_5$. The median weights of $\mathcal{T}_1^{\text{sub}}$ and $\mathcal{T}_2^{\text{sub}}$ are then compared, and $\mathcal{T}_1^{\text{sub}}$ is flagged as the subtree containing adversarial participants. The

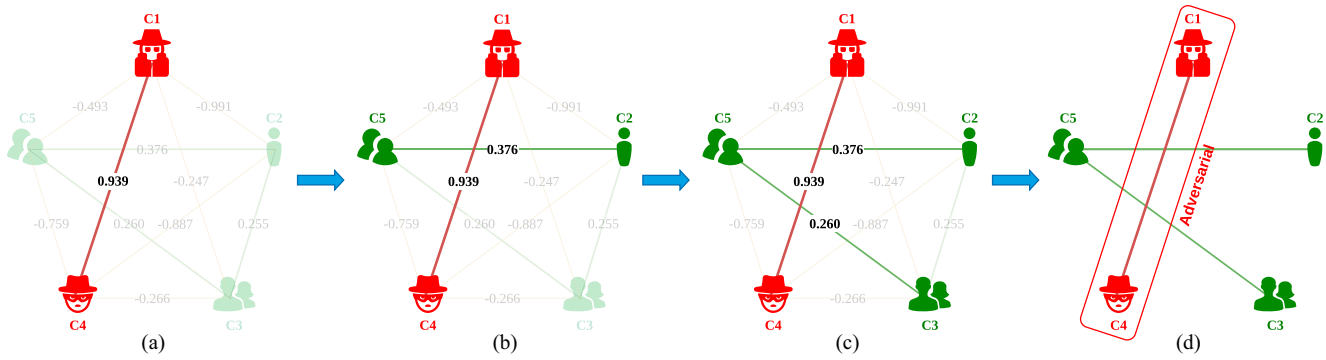

Fig. 3. Illustrating intermediate steps of the MST-AD algorithm for an FL setup with five clients. (a) Step 1. (b) Step 2. (c) Step 3. (d) Step 4.

Algorithm 2: MST-AD Algorithm.

Input: W^0
Output: W^*
At Server:
for $t = 1$ *to* T **do**
  for *each client* $c_i \in \mathcal{C}$ **do**
    $\delta_i^t \leftarrow$ `Client`(W^t) `// See Algorithm 1`
  $\mathcal{B} \leftarrow$ `MST`$(|\delta^t|)$ `// B: Set of benign clients`
  $W^{t+1} \leftarrow W^t + \frac{1}{\sum_{c_i \in \mathcal{B}} |\mathcal{D}_i|} \sum_{c_i \in \mathcal{B}} |\mathcal{D}_i| \delta_i^t$
return $W^* = W^{t+1}$

function `MST` $(|\delta^t|)$ **:**
  `//` δ^t `is a set of updates of all clients`
  $\mathcal{A} \leftarrow$ Compute_Correlation$(|\delta^t|)$
  $\mathcal{T} \leftarrow \emptyset$ `// Initializing a tree` $\mathcal{T}$
  $\mathcal{E} \leftarrow$ Set of sorted edges in decreasing order
  for $e \in \mathcal{E}$ **do** `// Iterating through edges`
    if *e does not form cycle in* $\mathcal{T}$ **then**
      $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{E}[i]$ `// Adding edges in` $\mathcal{T}$
  $\mathcal{T}_1^{sub}, \mathcal{T}_2^{sub} \leftarrow$ Remove lowest weighted edge from $\mathcal{T}$
  $\mathcal{M} \leftarrow \arg max(median(\mathcal{T}_1^{sub}), median(\mathcal{T}_2^{sub}))$
  return $\mathcal{B} = \mathcal{C} \setminus \mathcal{M}$

weights δ_1 and δ_4 are subsequently removed from the training process.

Theorem 1: Given a correlation matrix $\mathcal{A}$, let inequality 3 hold for any pair of clients, then Algorithm 2 returns the complete and correct set of colluding adversaries.

Proof: Let $\mathcal{E}$ be the set of edges ordered in decreasing order of their weights in the graph induced by $\mathcal{A}$. Assuming that the inequality $\rho_{ip} < \rho_{ij} < \rho_{pq}$ holds for any pair of adversaries $c_p, c_q \in \mathcal{M}$ and any pair of benign clients $c_i, c_j \in \mathcal{C} \setminus \mathcal{M}$, we can divide $\mathcal{E}$ into three contiguous subgroups. The first subgroup contains edges between pairs of adversaries, followed by the subgroup of edges between any pair of benign clients, and finally the last subgroup formed by the edges between benign and adversarial clients. Using the definition of MST 1, we need to select $(n-1)$ edges from $\mathcal{E}$ starting with the edges with maximum weights. Following the MST construction, we obtain a subtree comprising all the adversaries (edges with higher weights), another subtree consisting of all the benign clients (second group of edges in $\mathcal{E}$), and finally, one single edge (the one with lower weight in such a tree) connecting the two subtrees. By removing this edge, we get two subtrees: one whose vertices correspond to benign clients and the other whose vertices correspond to adversaries. This concludes the proof. ■

D. Density-AD Algorithm

Given the graph consisting of clients as nodes and correlation values as edges, we propose another attacker detection algorithm, density-AD, leveraging the concept of graph density. This algorithm utilizes the average edge density of the nodes instead of individual edges in the MST. This makes the algorithm more robust to edges with inconsistent weights compared to MST-AD. For a given graph $\mathcal{G}$, the density-AD algorithm essentially performs three steps.

1) *Remove Sparse Nodes From* $\mathcal{G}$: The density-AD algorithm first computes the density of the graph as follows:

$$\text{density}(\mathcal{G}) = \frac{2\sum_{i=1}^{n}\sum_{j=1}^{n}\rho_{ij}}{n(n-1)} \tag{4}$$

where ρ_{ij} is equal to $\mathcal{A}[i,j]$. Next, for each vertex $v \in \mathcal{G}$, if $\text{density}(\mathcal{G} \setminus v) > \text{density}(\mathcal{G})$ then we remove v (with its edges) and update $\mathcal{G} = \mathcal{G} \setminus v$. The removed vertex v is then added to a set of sparse vertices $\mathcal{V}$.

2) *Construct Two Subgraphs*: We form a graph $\mathcal{G}^{\text{sub}}_{\text{sparse}}$ by reconnecting the vertices of $\mathcal{V}$ using their original edges that were present in $\mathcal{G}$. Note that while the nodes in $\mathcal{V}$ were sparse in $\mathcal{G}$, the density of $\mathcal{G}^{\text{sub}}_{\text{sparse}}$ can be higher than the other subgraph $\mathcal{G}^{\text{sub}}_{\text{dense}} = \mathcal{G} \setminus \mathcal{V}$.

3) *Identify Subgraph With Adversarial Nodes:* When the attackers are in the minority, $\mathcal{G}^{\text{sub}}_{\text{sparse}}$ would contain the adversarial nodes; however, it would have only benign nodes if the adversaries are in the majority. Thus, we employ median edge weight to identify the adversarial nodes. In the density-AD, we flag the subgraph with a higher median edge weight as the subgraph comprising adversarial nodes.

Algorithm 3 illustrates all the steps of density-AD.

Example 2: We provide an example on the representative graph in Fig. 4 to show the process of the density-AD algorithm. We initiate the process with the removal of the first node c_1 and calculate the density of $\mathcal{G} \setminus c_1$ using (4). As the

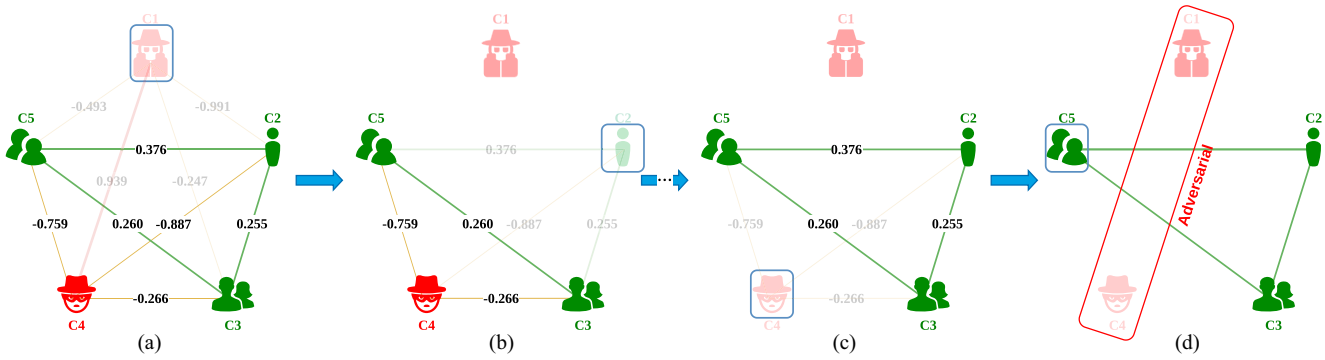

Fig. 4. Illustrating intermediate steps of the density-AD algorithm for an FL setup with five clients. (a) Step 1. (b) Step 2. (c) Step 3. (d) Step 4.

Algorithm 3: Density-AD Algorithm.

Input: W^0
Output: W^*
At Server:
for $t = 1$ *to* T **do**
  for *each client* $c_i \in \mathcal{C}$ **do**
    $\delta_i^t \leftarrow$ `Client`(W^t) `// See Algorithm 1`
  $\mathcal{B} \leftarrow$ `Density`$(|\delta^t|)$ `// B: Benign clients`
  $W^{t+1} \leftarrow W^t + \frac{1}{\sum_{c_i \in \mathcal{B}} |\mathcal{D}_i|} \sum_{c_i \in \mathcal{B}} |\mathcal{D}_i| \delta_i^t$
return $W^* = W^{t+1}$

function `Density`$(|\delta^t|)$**:**
  `//` δ^t `is a set of updates of all clients`
  $\mathcal{A} \leftarrow$ Compute_Correlation$(|\delta^t|)$
  $\mathcal{V} \leftarrow \emptyset$ `// Initializing list of sparse nodes`
  for $v \in \mathcal{G}$ **do** `// Iterating through vertices`
    if $density(\mathcal{G} \setminus v) > density(\mathcal{G})$ **then**
      $\mathcal{G} \leftarrow \mathcal{G} \setminus v$ `// Updating the graph`
      $\mathcal{V} \leftarrow \mathcal{V} \cup v$ `// Updating the sparse list`
  Construct a sub-graph $\mathcal{G}_{sparse}^{sub}$ using $\mathcal{V}$ nodes
  Construct a sub-graph $\mathcal{G}_{dense}^{sub}$ using $\mathcal{G} \setminus \mathcal{V}$ nodes
  $\mathcal{M} \leftarrow \arg max\big(median(\mathcal{G}_{dense}^{sub}), median(\mathcal{G}_{sparse}^{sub})\big)$
  return $\mathcal{B} = \mathcal{C} \setminus \mathcal{M}$

density increases from -0.181 to -0.17, the node c_1 is permanently removed from $\mathcal{G}$ and appended into $\mathcal{V}$. Next, the removal of c_2 and c_3 individually decreases the density to -0.256 and -0.408, respectively, and thus both nodes are kept in $\mathcal{G}$. Upon removal of c_4, the density increases to 0.297, making c_4 a sparse node and thus removed from $\mathcal{G}$ and appended to $\mathcal{V}$. Finally, comparing the median weight of $\mathcal{G}_{\text{dense}}^{\text{sub}} = \mathcal{G} \setminus \{c_1, c_4\} = \{c_2, c_3, c_5\}$ with that of $\mathcal{G}_{\text{sparse}}^{\text{sub}} = \{c_1, c_4\}$, density-AD annotates $\mathcal{G}_{\text{sparse}}^{\text{sub}}$ as the subgraph of the targeted attackers c_1 and c_4.

1) Node Removal Criteria: A node $v \in \mathcal{G}$ can be removed if one of two criterion is satisfied: *C1*: $\text{density}(\mathcal{G} \setminus v) > \text{density}(\mathcal{G})$, or *C2*: its corresponding edge density is lower than the edge density of the original graph. Density-AD uses the first criterion. Both criteria give the same results as they are mathematically equivalent, as stated in Theorem 2.

Theorem 2: Given graph $\mathcal{G}$, let $k \in [0, n)$ be the number of sparse nodes already removed from $\mathcal{G}$, then the $(k+1)$th node is sparse if the following condition holds:

$$\frac{2[S - \sum_{i=1}^{n}\sum_{j=1}^{k}\rho_{i,j} + \sum_{i=1}^{k}\sum_{j=1}^{i}\rho_{i,j}]}{(n-k)(n-k-1)} > \frac{\sum_{i=1}^{n}\rho_{i,k+1} - \sum_{i=1}^{k+1}\rho_{i,k+1}}{(n-k-1)}$$

where S is the sum of edge weights, i.e., $S = \sum_{i=1}^{n}\sum_{j=1}^{n}\rho_{ij}$.

Proof: Let the first k nodes (satisfying the inequality) be removed from the graph. Knowing that $\mathcal{G}$ is a complete graph, the removal of kth node results in the removal of $(n-k)$ corresponding edges. Thus, the total number of edges removed upon removing k vertices is $(n-1)+(n-2)+\ldots+(n-k) = nk - k(k+1)/2$. And their corresponding total weight is $\sum_{i=1}^{n}\sum_{j=1}^{k}\rho_{i,j} - \sum_{i=1}^{k}\sum_{j=1}^{i}\rho_{i,j}$. To this end, the density of the graph after the removal of k nodes can be given as follows:

$$\frac{2\left\{S - \left(\sum_{i=1}^{n}\sum_{j=1}^{k}\rho_{i,j} - \sum_{i=1}^{k}\sum_{j=1}^{i}\rho_{i,j}\right)\right\}}{(n-k)(n-k-1)}.$$

Now, the next node $k+1$ can be removed only if

$$2 \times \frac{S - \sum_{i=1}^{n}\sum_{j=1}^{k}\rho_{i,j} + \sum_{i=1}^{k}\sum_{j=1}^{i}\rho_{i,j}}{(n-k)(n-k-1)} < 2 \times \frac{S - \sum_{i=1}^{n}\sum_{j=1}^{k+1}\rho_{i,j} + \sum_{i=1}^{k+1}\sum_{j=1}^{i}\rho_{i,j}}{(n-k-1)(n-k-2)} \quad (5)$$

where the LHS denotes the average edge weight before the $(k+1)$th node is removed, and the RHS denotes the average edge weight once $(k+1)$th node is removed. Upon simplifying (5), we get

$$(n-k-2)\left[S - \sum_{i=1}^{n}\sum_{j=1}^{k}\rho_{i,j} + \sum_{i=1}^{k}\sum_{j=1}^{i}\rho_{i,j}\right] < (n-k)\left[S - \sum_{i=1}^{n}\sum_{j=1}^{k}\rho_{i,j} - \sum_{i=1}^{n}\rho_{i,k+1} + \sum_{i=1}^{k}\sum_{j=1}^{i}\rho_{i,j} + \sum_{j=1}^{k+1}\rho_{k+1,j}\right].$$

By substituting $\sum_{i}^{k+1}\rho_{i,k+1}=\sum_{j}^{k+1}\rho_{k+1,j}$ we obtain

$$\frac{2\left[S-\sum_{i=1}^{n}\sum_{j=1}^{k}\rho_{i,j}+\sum_{i=1}^{k}\sum_{j=1}^{i}\rho_{i,j}\right]}{(n-k)(n-k-1)} > \frac{\sum_{i=1}^{n}\rho_{i,k+1}-\sum_{i=1}^{k+1}\rho_{i,k+1}}{(n-k-1)} \tag{6}$$

where the LHS is graph density before the removal of $(k+1)$th node and the RHS denotes the average edge weight of $(k+1)$th node. ■

2) Complete Separation of Targeted Attackers: To accurately identify the attackers, we assume that (3) holds and then prove the complete separation of adversarial and benign clients from $\mathcal{G}$.

Theorem 3: Given a correlation matrix $\mathcal{A}$ and the inequality in (3) being held for any pair of clients, the density-AD Algorithm 3 provides a complete separation of all the attackers from the benign clients.

Proof: For convenience, let ρ_{AA}, ρ_{AB}, and ρ_{BB} be the weight correlations between different clients, where A and B represent attacker and benign clients, respectively.

Case 1: Assuming that the first $k\in[0,1)$ nodes that were identified as sparse nodes and removed from the graph were adversarial nodes. Using the criterion for removal of $(k+1)$th node as defined in (6), we obtain

$$2\times\frac{S-(n-m)k\rho_{AB}-\frac{k(2m-1-k)}{2}\rho_{AA}}{(n-k)} > \frac{(n-m)\rho_{AB}+(m-k-1)\rho_{AA}}{1}.$$

Also, for a graph with m adversaries, there are $\binom{m}{2}$ adversarial–adversarial edges, $\binom{n-m}{2}$ benign–benign edges, and $(n-m)m$ adversarial–benign edges. Thus, the cumulative sum of the edges $S=\binom{m}{2}\rho_{AA}+\binom{n-m}{2}\rho_{BB}+(n-m)m\rho_{AB}$. Thus, the above inequality becomes

$$\begin{aligned}2\Bigg(&\frac{m(m-1)}{2}\rho_{AA}+\frac{(n-m)(n-m-1)}{2}\rho_{BB}\\&+m(n-m)\rho_{AB}-nk\rho_{AB}\\&+mk\rho_{AB}-\frac{k(2m-k-1)}{2}\rho_{AA}\Bigg)\\&>(n-k)((n-m)\rho_{AB}+(m-k-1)\rho_{AA}).\end{aligned}\tag{7}$$

Multiplying the relevant terms, we get

$$\begin{aligned}&m^2\rho_{AA}-m\rho_{AA}+(n-m)(n-m-1)\rho_{BB}\\&+2mn\rho_{AB}-2m^2\rho_{AB}-2nk\rho_{AB}+2mk\rho_{AB}\\&-2mk\rho_{AA}+k^2\rho_{AA}+k\rho_{AA}\\&>n^2\rho_{AB}-nm\rho_{AB}-nk\rho_{AB}+mk\rho_{AB}\\&+nm\rho_{AA}-nk\rho_{AA}-n\rho_{AA}-mk\rho_{AA}\\&+k^2\rho_{AA}+k\rho_{AA}.\end{aligned}\tag{8}$$

After simplification, we have

$$\begin{aligned}&\rho_{AA}(m^2-m-mk+n+nk-nm)\\&+\rho_{AB}(3nm-n^2-2m^2-nkv+mkv)\\&+\rho_{BB}(n-m)(n-m-1)>0.\end{aligned}\tag{9}$$

Factorization yields

$$\begin{aligned}&\rho_{AA}(n-m)(k+1-m)+\rho_{AB}(n-m)(2m-n-k)\\&+\rho_{BB}(n-m)(n-m-1)>0.\end{aligned}$$

Eliminating the term $(n-m)$ does not affect the inequality as $(n-m)>0$ where $m=|\mathcal{M}|, n=|\mathcal{C}|, \mathcal{M}\subset\mathcal{C}$. Thus, eliminating the term and further simplifying yields

$$\frac{(n-m)-1}{m-k-1}>\frac{\rho_{AA}-\rho_{AB}}{\rho_{BB}-\rho_{AB}}.\tag{10}$$

Provides us with the condition for the removal of $(k+1)$th adversarial nodes when the k adversarial nodes are removed. We also mention that as $\rho_{AA}>\rho_{AB}\ \forall\rho_{AA},\rho_{AB}$, the set of adversaries is separated from the graph only when $(n-m)-1>m-k-1$. Thus, for the set of adversaries to be removed, the number of benign agents should be more than the number of adversarial agents remaining in the graph. As decreasing k increases the RHS, the limiting case occurs when the first node is removed, and the necessary and sufficient condition for the removal of the first adversarial client is as follows:

$$\frac{(n-m)-1}{m-1}>\frac{\rho_{AA}-\rho_{AB}}{\rho_{BB}-\rho_{AB}}.\tag{11}$$

By satisfying (11), the density-AD algorithm isolates the first adversarial node, and thereby, through induction, the entire set of adversarial nodes iteratively.

Case 2: Similarly, in a scenario where the first k nodes removed are benign, the criterion for the removal of $(k+1)$th benign node from the graph becomes

$$2\times\frac{S-mk\rho_{AB}-\frac{k(2(n-m)-k-1)}{2}\rho_{AB}}{(n-k)} > \frac{(m\rho_{AB}+((n-m)-k-1)\rho_{BB})}{1}.$$

Using the same expansion of S as in (7), we obtain

$$\begin{aligned}2\Bigg(&\frac{m(m-1)}{2}\rho_{AA}+\frac{(n-m)(n-m-1)}{2}\rho_{BB}\\&+m(n-m)\rho_{AB}-mk\rho_{AB}\\&-\frac{k(2(n-m)-k-1)}{2}\rho_{BB}\Bigg)\\&>(n-k)(m\rho_{AB}+((n-m)-k-1)\rho_{BB}).\end{aligned}$$

Simplifying the expression, we get

$$\begin{aligned}&m^2\rho_{AA}-m\rho_{AA}+(n-m)^2\rho_{BB}-(n-m)\rho_{BB}\\&+2m(n-m)\rho_{AB}-2mk\rho_{AB}-2k(n-m)\rho_{BB}+k^2\rho_{BB}\\&+k\rho_{BB}>nm\rho_{AB}-km\rho_{AB}+n(n-m)\rho_{BB}\\&-k(n-m)\rho_{BB}-nk\rho_{BB}-n\rho_{BB}+k^2\rho_{BB}+k\rho_{BB}.\end{aligned}$$

Bringing the terms together

$$\begin{aligned}&m^2\rho_{AA}-m\rho_{AA}+m\rho_{AB}[2(n-m)-k-n]>\\&-\rho_{BB}[(n-m)^2-(n-m)-k(n-m)\\&-n(n-m)+nk-n].\end{aligned}$$

Further simplification yields

$$\begin{aligned}&m(m\rho_{AA}-\rho_{AA})+m\rho_{AB}[2(n-m)-k-n]\\&+\rho_{BB}[m(m+k+1)-nm]>0.\end{aligned}$$

Eliminating the term m as it is greater than 0 and simplifying yields

$$\frac{(n-m)-k-1}{(m-1)} < \frac{(\rho_{AA}-\rho_{AB})}{(\rho_{BB}-\rho_{AB})} \tag{12}$$

which provides the condition for the detection and separation of the benign agents from the setup. Similar to (10), the value of $\rho_{AA} > \rho_{AB}, \forall \rho_{AA}, \rho_{AB}$ implies that the inequality becomes more prominent when $(n-m)-k-1 < m-1$ which implies the set of adversarial agents to be more than the set of benign agents left in the graph. We further mention that, as in (10), the inequality becomes more prominent as k increases. Hence, we describe the necessary and sufficient conditions for the separation of benign participants as follows:

$$\frac{(n-m)-1}{(m-1)} < \frac{(\rho_{AA}-\rho_{AB})}{(\rho_{BB}-\rho_{AB})} \tag{13}$$

satisfying which the algorithm identifies and separates the entire set of benign agents from the setup.

As either of (11) or (13) is true for a given setup, the density-AD algorithm thus separates the set of adversarial and benign clients during training. The set containing the adversarial clients can be identified by comparing the median edge weights of the separated nodes and the remaining graph. We do mention that while the proof assumed on the weights for a particular type of correlations being the same, experimental results validate the effectiveness of the density-AD algorithm on setups where the weights are heterogeneous. ■

E. Complexity Analysis

We assume an FL setup with n clients each submitting weight updates with a dimensionality d. The complexity for computing the complexity between a pair of participants is of the order of $O(d)$. Among n clients, we compute a total of $\binom{n}{2}$ correlations. Thus, the overall complexity for generating the adjacency matrix $\mathcal{A}$ is of the order $O(n^2 \cdot d)$. This is consistent with the computational complexity mentioned in other correlation-based algorithms such as [12] and [34].

After obtaining the adjacency matrix $\mathcal{A}$, we perform the MST-AD and density-AD algorithm on the setup. The computational complexity for the MST-AD algorithm on a graph $\mathcal{G}$ with n nodes is of the order of $O(n \cdot log(n))$. For density-AD, the computational complexity is of the order of $O(n^2)$ as $n-1$ edges are considered for each of the n nodes. Therefore, the computational complexity of FedDOT is dominated by $O(n^2 \cdot d)$, which gives the per-round complexity of the framework.

F. Convergence Analysis

As the FedDOT framework removes adversarial weight updates, the loss function effectively becomes $l(W) = 1/|\mathcal{B}| \sum_{c_i \in \mathcal{B}} l_i(W)$. For convergence analysis, we mention that the client loss function $l_i(\cdot)$ satisfies the following standard assumptions:

Assumption 1: The loss function is L-smooth and lower bounded such that $l_i(W) \leq l_i(W') + (W-W')^T \nabla l_i(W) + (L/2)||W-W'||^2, \forall c_i \in \mathcal{B}$.

Assumption 2: The loss function is β-convex such that $l_i(W) \geq l_i(W') + (W-W')^T \nabla l_i(W) + (L/2)||W-W'||^2$, $\forall c_i \in \mathcal{B}$, where $W, W' \in \mathbb{R}^d$ and $||\cdot||$ is the euclidean norm.

Inspired by the approach in [40], we give the convergence of the global model as follows:

$$l(W^t) - l(W^*) \leq \epsilon$$

where W^* is the optimal solution and $\epsilon > 0$ is the error margin. We define the updating of the local weights for the clients as $W^{t+1} = W^t - \eta \nabla l_i(W^t)$ for the clients. We further mention that the global loss function has the same Hessian matrix and shows convexity and smoothness similar to local loss functions. Thus, we use the gradient method described in [41], to define the convergence of the global model as follows:

$$l(W^t) - l(W^*) \leq (1-\theta)^t (l(W^0) - l(W^*))$$

where W^0 is the model at $t=0$ and θ depends on the learning rate η and the nature of the function l. Thus, (14) analyzes the convergence of the server model W^t to the optimum W^*. Furthermore, the value of t required for the convergence of the global model to within the error value ϵ is derived as follows:

$$T = \frac{1}{\theta} \log \left(\frac{l(W^0) - l(W^*)}{\epsilon} \right)$$

which estimates the optimum number of training rounds during performance evaluation in Section V-C2.

V. Performance Evaluation

This section evaluates the effectiveness of FedDOT and reports the results obtained from extensive experiments involving a variety of targeted attackers. Later, we show that FedDOT is superior to existing popular approaches.

A. Experimental Setup

Considering an image classification task in this work, we build a deep neural network consisting of a 2 convolutional neural network [42] layers followed by three fully-connected layers. We use three existing benchmark datasets, including MNIST [37], fashion-MNIST (FMNIST in short) [43], and CIFAR-10 [44]; each of which consists of 60 000 training and 10 000 testing grey scale images divided equally into ten classes. The samples from each dataset are partitioned into $n = 50$ disjoint subsets using a Dirichlet distribution with parameter $\alpha = 0.9$ [10] (for non-IID data). Each subset is assigned to a single client. For easy reading, the attack scenarios are denoted by $A{-}m$ where A is a shorthand for the term "attack" and m is the percentage of attackers to the total number of clients. For instance, $A-10$ refers to the scenario with 10% of the total clients as collusion-targeted attackers.

Attacks: We simulate the considered attacks as follows.

1) *Single-Label Flipping (SF)*: This is bidirectional label flipping in which the labels of samples of two targeted classes are flipped or swapped. The attacker clients flip "0" and "1" in the MNIST dataset, "T-Shirt" and "Trouser" in the FMNIST dataset, and "Airplane" and "Automobile" in the CIFAR-10 dataset, respectively.

Fig. 5. Backdoor patterns for fashion-MNIST dataset.

2) *Multilabel Flipping (MF):* Multiple labels are flipped by the attackers. For MNIST dataset, we replace labels for digits "5" and "9" with "7" and "1," respectively. Similarly, for the FMNIST dataset, the labels "Sandal" and "Ankle Boot" are replaced with "Sneaker" and "Trouser," respectively. Finally, for the CIFAR-10 dataset, the labels "Dog" and "Truck" are flipped to "Horse" and "Automobile" respectively.
3) *Backdoor:* Each attacker incorporates a trigger pattern ("= =") into the targeted images (samples) of the local data and replaces their labels with "8" for the MNIST, "Bag" for the FMNIST, and "Ship" for the CIFAR-10 dataset, respectively, thereby inducing a collusion attack. For each attacker, this corruption is added to 50% of the local data. We include a version of backdoor attacks in which the position of the pattern is changed across all the attacked images. In addition, the width of the trigger pattern varies to introduce variation in the attacks. We show these variations in Fig. 5 for the Fashion-MNIST dataset.

We further describe a form of attack (referred to as *Compound Attack*) where each adversary can choose any of the above three attacks within a single FL setup, thereby creating a scenario of mixed attacker existence at every round. We assume that the number of adversaries performing either of these attacks is equal.

B. Performance Metrics

We employ the following metrics to quantify the performance.

1) *Training Loss*: The decay of the loss function with respect to the training rounds.
2) *Test Accuracy*: The proportion of correctly classified samples in the test set.
3) *Attack Success Rate (ASR)*: The proportion of the poisoned samples (either with flipped labels or backdoor patterns) misclassified into the target label [12].
4) *Earliest Detection Round (ED) and False Positives (FP)*: The first training round at which all the attackers got detected successfully and the number of benign participants incorrectly identified as adversaries in that particular round.
5) *Confusion Matrix*: It depicts the confusion observed by the classification model across all test samples of all the classes.

C. Experimental Results

We report results on test data while comparing our algorithms with competitive detection algorithms. We consider FoolsGold [12], CONTRA [34], FedBAP [45], and FLAME [31] algorithms for comparison. Our experiments include federated averaging (FedAvg) [1] as a baseline.

TABLE I
ACCURACY(IN %) FOR MNIST DATASET FOR NO ATTACK AND OVERWHELMING A-70 ATTACK

Algorithm	No Attack	Single-LF	Multi-LF	Backdoor
MST-AD	93.11	92.39	92.12	92.34
Density-AD	93.1	92.97	92.41	92.19

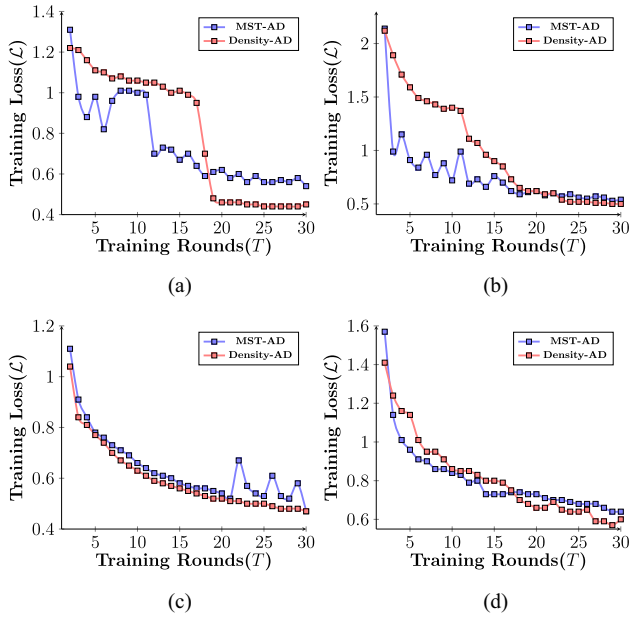

Fig. 6. Training loss ($\mathcal{L}$) over FL rounds (T) for $A-70$ attack using fashion-MNIST dataset. (a) SF attack. (b) MF attack. (c) Backdoor attack. (d) Compound attack.

1) No Attack Scenario: To ascertain the robustness of the setup, we run basic experiments with no attacker participants. This confirms the efficacy of the precursory check in a setup with no attackers. We first simulate a setup with no attackers (A-0 attack) and then perform the overwhelming A-70 attack. The corresponding accuracy values for the MNIST dataset are mentioned in Table I for different attacks. We observe that the change in accuracy is minimal for the A-0 attack and there is negligible accuracy drop for the A-70 attacks. The support-based check allows clean setups to pass unfiltered whereas setups with adversaries are identified and the adversarial participants are isolated by the AD algorithms.

2) Training Loss Over FL Rounds: First, we report the training loss for density-AD and MST-AD algorithms for a severe attack scenario (with 70% attackers abbreviated as $A-70$ attack) using fashion-MNIST, in Fig. 6. To avoid replication, we omit similar plots for low-severity level cases. We observe that the model starts converging after 25th round. density-AD performs slightly better than the MST-AD algorithm. A sharp decline after the 20th round

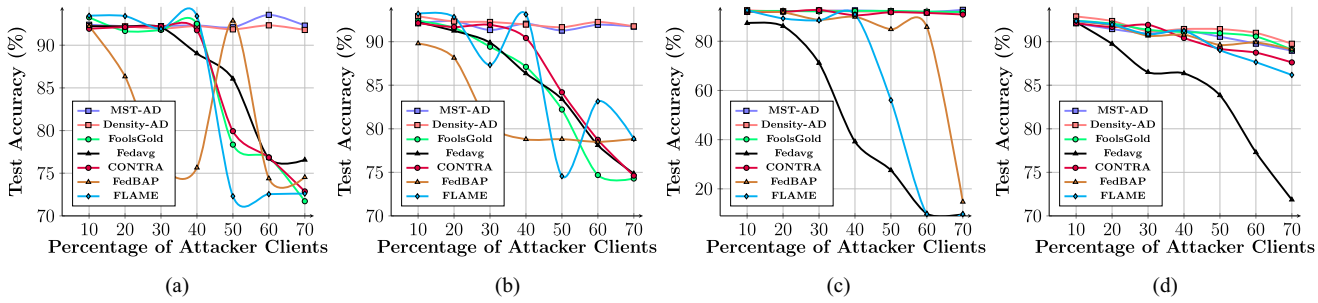

Fig. 7. Test accuracy using MNIST dataset. (a) SF attack. (b) MF attack. (c) Backdoor attack. (d) Compound attack.

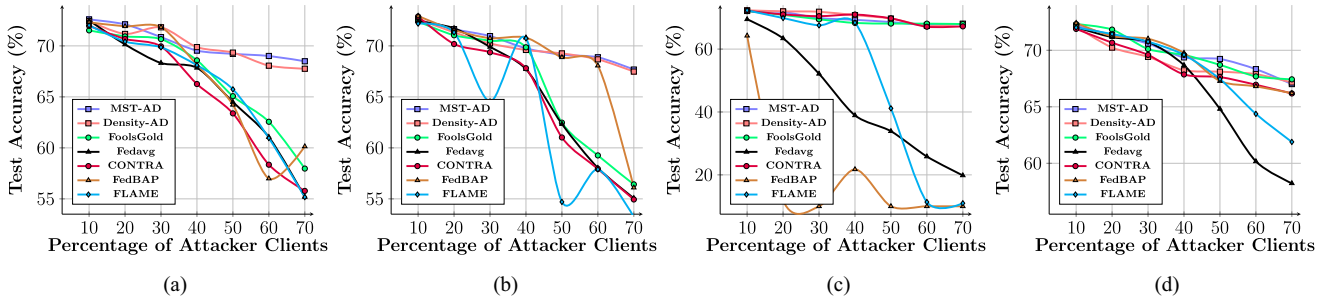

Fig. 8. Test accuracy using CIFAR-10 dataset. (a) SF attack. (b) MF attack. (c) Backdoor attack. (d) Compound attack.

is also attributed to the correct detection of all adversaries. Furthermore, the fluctuations in the loss curve, especially for the MST-AD mainly caused by certain adversaries escaping detection for the particular round. By analyzing these results, we set the number of FL rounds at 30 in all our subsequent experiments.

3) Impact of Colluding Attackers on Test Accuracy: Next, we report the test accuracy of the proposed algorithms with a varying percentage of attackers in Fig. 7 for MNIST and in Fig. 8 for CIFAR-10. Our algorithms show consistent performance for single- and multilabel flipping attacks compared to FoolsGold, FedAvg, and CONTRA algorithms, especially when the percentage of attackers is greater than 40%. However, due to the diversity of the CIFAR-10 dataset, we notice fluctuations and an overall decrease in accuracy for our algorithms, which is minimal compared to the existing algorithms. The FLAME and FedBAP algorithms also show robust performance on the CIFAR dataset for single- and multilabel flipping attacks when the number of adversaries is lower. However, they showcase a larger drop in performance as the number of adversarial participants increases.

Even under the backdoor attack scenario, FoolsGold and CONTRA managed to offer comparable accuracy to our algorithms. GeoMed performs better than FedAvg up to attackers less than $30\% - 40\%$, but they both perform worse than FoolsGold and CONTRA due to incorrect utilization of adversarial weight updates over benign updates. FLAME maintains higher accuracy in cases where the attacker clients are in the minority; however, its performance degrades substantially when the number of attackers is $> 50\%$. FedBAP also showcases higher performance when the number of attackers is reduced. However, as the number of attackers increases, FedBAP suffers from much greater performance degradation than the proposed algorithms. Further, we also report the results for a compound attack scenario with different types of attackers in part (d) of Figs. 7 and 8. It is interesting to see that FoolsGold and CONTRA achieve quite similar accuracy to ours, as they do in the backdoor attack case. Here, a slight drop may be noticed for all the algorithms when the adversaries overwhelm the benign clients. Additionally, FedBAP showcases improved performance comparable to the proposed algorithms, but it also suffers from a similar performance drop. However, MST-AD and density-AD still work better than FoolsGold, CONTRA, and FLAME while outperforming the FedAvg algorithm substantially.

4) Analyzing ASR: This section reports the ASR results with varying numbers of attackers in Table II. Under SF attack, regardless of the percentage of attackers, the proposed algorithms offer the best possible defense with ASR exactly 0 for the MNIST dataset and less than 4 for the fashion-MNIST dataset, while the strong prior algorithms (FoolsGold and CONTRA) could not detect even a single attacker (that is, ASR $= 100$) in case of A-70 attack for MNIST dataset. FLAME has been able to maintain a lower value of ASR in cases where the fraction of adversaries is less than 50%, but the ASR increases when the fraction of adversaries is greater than 50%. However, the ASR for the proposed algorithms is higher for the CIFAR-10 dataset ($\sim 8\%$) due to its higher complexity. Still, the proposed algorithms vastly outperform FoolsGold and CONTRA, both of which show a significant improvement compared to

TABLE II
ASR WITH VARYING (10%–70%) NUMBER OF ATTACKERS USING MNIST, FASHION-MNIST AND CIFAR-10 DATASETS

Dataset	Attacks	MST-AD			Density-AD			FoolsGold			FedAvg			FedBAP			CONTRA			FLAME		
		SF	MF	Back-door	SF	MF	Back-door	SF	MF	Back-door	SF	MF	Back-door	SF	MF	Back-door	SF	MF	Back-door	SF	MF	Back-door
MNIST	A-10	0	0.6	0.26	0	0.6	0.39	0	0.4	0.39	0	0.6	11.25	5.43	4.32	0.22	0	0	0.31	0.13	0.35	0.87
	A-20	0	0.23	0.44	0	0.54	0.14	0	0.6	0.33	0.25	1.5	28.68	15.38	26.17	0.35	0	0	0.49	0.21	2.45	5.82
	A-30	0	1.67	0.33	0	1.9	0.26	0	2.58	0.45	1.67	6.55	61.39	91.35	81.12	4.68	0.57	0	0.56	6.28	19.81	10.65
	A-40	0	1	0.56	0	1.25	0.48	0	4.8	0.28	34	23.75	73.29	94.29	93.57	3.46	1.9	2.31	0.79	0.59	0.65	6.34
	A-50	0	0.88	0.28	0	1.67	0.52	3.4	9.03	0.39	25.1	37	76.52	1.02	94.61	17.49	7.88	15.79	0.63	100	96.54	46.14
	A-60	0	0.83	0.19	0	0.81	0.44	77.4	13.7	0.50	100	64.95	90.1	97.15	97.16	5.82	42.94	26.78	0.28	100	51.24	90
	A-70	0	0.71	0.95	0	0.71	0.32	100	12.2	0.40	96.39	83.2	100	98.91	95.45	90	97.4	28.1	0.55	100	61.7	90
FMNIST	A-10	3	0.34	0.95	3.5	1.95	0.80	4.52	3.65	0.97	2.5	4.8	8.83	10.98	11.87	3.51	0.31	5.43	0.66	0.36	0.67	1.13
	A-20	1.5	0.54	0.80	1	1.65	0.68	1.52	2.4	0.94	3	14.15	28.18	7.17	6.5	1.99	2.56	6.21	0.46	11.72	2.61	2.96
	A-30	1	7.8	0.84	1.67	8	0.60	0.84	4.67	0.86	2.3	16.45	59.56	77.28	74.56	3.84	7.01	6.67	1.19	18.47	3.78	19.23
	A-40	0.5	6.3	0.85	1.63	7.1	0.76	6	10.9	0.95	8.5	25.15	10.19	90.16	94.21	11.14	21.93	8.12	0.92	2.08	99.87	10.91
	A-50	1.1	5.78	0.20	0.81	6.67	0.60	43.2	9.44	0.83	82.5	53.1	16.09	94.02	98.76	30	46.3	10.88	0.89	28.71	62.51	71.44
	A-60	1.83	7.75	1.11	0.54	2.8	0.57	66.15	14.16	0.93	67.9	84.1	89.92	94.37	88.64	67.2	67.41	19.63	0.97	11.41	18.14	90
	A-70	1	3.7	0.60	0.57	3.14	0.57	76.3	13.57	0.70	71.65	89.7	99.99	90.22	91.24	82	46.95	18.92	1.04	58.76	100	90
CIFAR	A-10	4.56	5.44	1.9	5.31	4.14	2.31	5.22	5.14	2.44	6.78	7.45	2.19	3.41	0.57	0.88	5.78	6.11	2.11	3.75	4.12	1.23
	A-20	6.87	7.56	2.34	5.63	5.12	1.46	9.39	8.01	2.33	8.9	8.18	6.47	5.9	7.59	8.11	8.99	7.49	1.98	10.67	9.81	4.16
	A-30	6.54	7.49	2.59	5.91	4.96	3.13	11.28	14.79	2.76	11.45	9.98	18.57	13.39	9.09	10.01	11.31	9.85	2.31	13.22	41.09	17.87
	A-40	8.11	6.97	3.15	6.13	5.1	3.2	16.99	15.45	2.77	16.16	13.97	23.11	16.21	18.16	16.6	13.47	13.22	3.04	21.68	11.72	19.56
	A-50	5.6	5.41	3.41	6.65	6.13	3.69	20.56	17.61	3.16	34.19	36.61	54.28	56.81	41.57	69.22	16.51	17.61	3.46	38.48	89.7	62.39
	A-60	4.98	7.34	3.88	7.78	8.1	3.07	16.13	18.89	3.9	67.76	65.44	79.22	77.43	75.46	79.99	21.36	18.9	2.98	55.19	78.36	90
	A-70	8.47	6.88	3.67	7.76	7.55	3	26.98	21.02	3.59	88.93	79.79	86.7	81.15	82.3	88.76	23.03	18.64	3.69	88.9	100	90

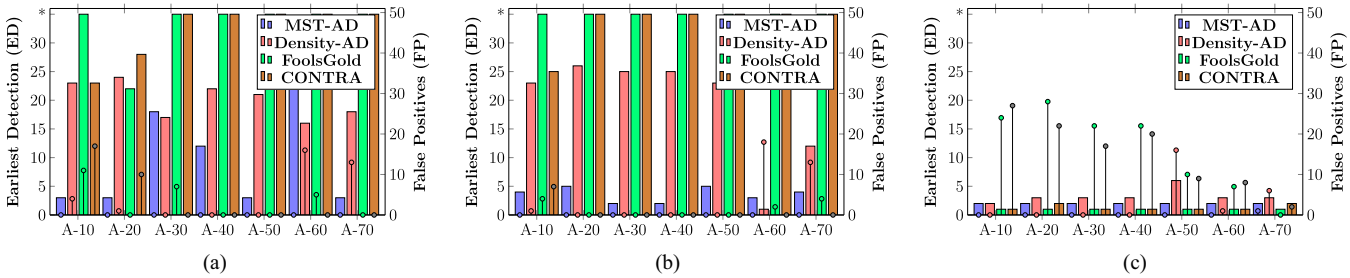

Fig. 9. ED and FP results using fashion-MNIST dataset. Bar plots represent ED, and comb plots (a line with a circle on the top) correspond to FP. On the left y-axis, the "$*$" depicts the scenarios where the algorithm could not detect the entire set of adversaries. (a) For SF attack. (b) For MF attack. (c) For Backdoor attack.

the MNIST and fashion-MNIST datasets. Under MF attack, though FoolsGold and CONTRA managed to reduce the ASR significantly for the MNIST and fashion-MNIST datasets, but could not perform close to MST-AD and density-AD; however, these algorithms showed tremendous improvement (with almost the same ASR $< 1\%$ as ours) against backdoor attackers for all three datasets. In addition, the proposed algorithms show an improvement in performance on the CIFAR-10 dataset for the backdoor attacker scenario. FedBAP underperforms on the SF and MF attacks for all three datasets. However, it shows mitigated ASR for backdoor attacks when the fraction of attackers is $< 40\%$. Note that other existing algorithms, FedAvg, GeoMed, and FLAME, remained poor performers in all scenarios. It is important to note that the proposed algorithms are capable enough to control the ASR (always less than 8%), indicating their effectiveness against overwhelming targeted attackers.

5) ED and FP: The earliest round (ED), when all attackers were detected, and the number of benign participants falsely flagged as adversaries (FP) are also two crucial evaluation metrics for a defense mechanism; we report their values in Figs. 9 and 10 using the fashion-MNIST and CIFAR-10 datasets, respectively. Here, the discussions on FedAvg, FedBAP, and FLAME algorithms are omitted as they focus on robustness, not the detection of adversaries. Under SF and MF attacks, the proposed algorithms are very effective as they can segregate all the attackers well before the last round (30th) whereas FoolsGold and CONTRA could not do so up to 30th round as represented by "$*$" mark, which means they would need many more rounds to detect the attackers, and meanwhile, the global model would keep aggregating the poisonous local models. However, both algorithms are able to keep the number of FP fairly low, and even in the cases A-60 and A-70, they outperform density-AD for the fashion-MNIST dataset. The performance of the proposed algorithms worsened for the CIFAR-10 dataset. However, both proposed algorithms once again outperformed FoolsGold and CONTRA in the early detection metric. In contrast, under a backdoor attack [Figs. 9(c) and 10(c)], FoolsGold and CONTRA achieve competitive results for ED but fail to maintain a fair number of FP values as they wrongly exclude benign participants during aggregation, which the proposed algorithms can avoid.

6) Confusion Matrices: Finally, in Figs. 11 and 12, we report confusion matrices for only the SF attack for the case of overwhelming attackers ($A-70$). In the interest of space, we do not include the matrices for various other settings. We see that FoolGold could not correctly distinguish the images of flipped

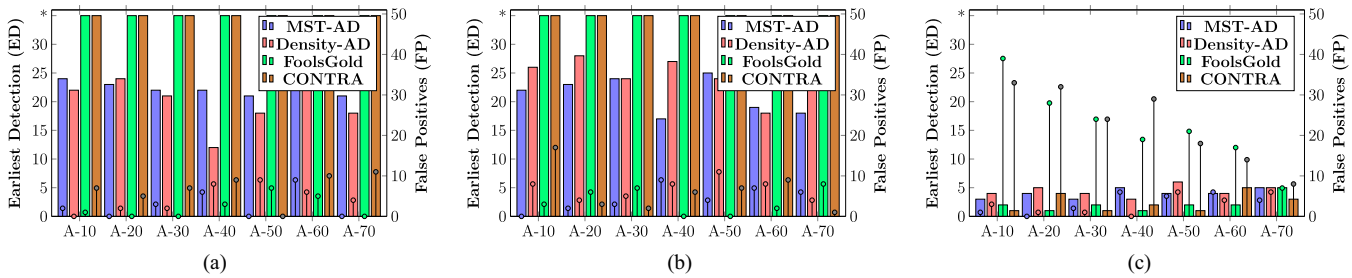

Fig. 10. ED and FP results using CIFAR dataset. Bar plots represent ED, and comb plots (a line with a circle on the top) correspond to FP. On the left y-axis, the "$*$" depicts the scenarios where the algorithm could not detect the entire set of adversaries. (a) For SF attack. (b) For MF attack. (c) For Backdoor attack.

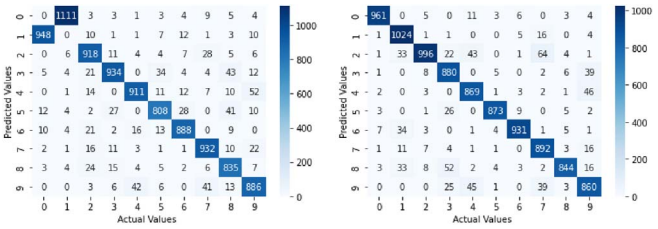

Fig. 11. Confusion matrices for the case of $A-70$ under SF attack using MNIST dataset. (a) FoolsGold. (b) Density-AD.

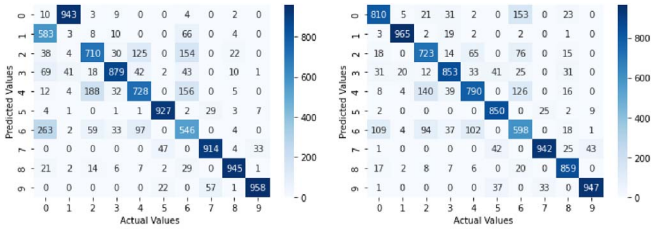

Fig. 12. Confusion matrices for the case of $A-70$ under SF attack using fashion-MNIST dataset. (a) FoolsGold. (b) Density-AD.

labels, i.e., between 0 and 1, whereas the proposed algorithms do not get stuck in any such confusion.

D. Ablation Study on Large Scale Dataset

To ascertain the performance of the proposed algorithm on larger datasets, we perform an ablation study using a more diverse dataset, Kuzushiji-49 (K-49) [46]. The K-49 dataset comprises of $270,912$ grayscale images of Japanese Hiragana characters of 28×28 sized images divided into 49 classes. We resize the images into 32×32 images and distribute the training samples across 50 clients following a Dirichlet data distribution with $\alpha = 0.9$. We perform the most extreme case (overwhelming $A - 70$) attack for both label flipping and backdoor attacks. We report the accuracy and ASR for both density-AD and MST-AD algorithms in Table III. The proposed algorithms

TABLE III
RESULTS FOR THE K-49 DATASET UNDER **A-70** ATTACK

Algorithm	SLF		MLF		Backdoor	
	Acc	ASR	Acc	ASR	Acc	ASR
Density-AD	80.24	0.46	80.7	1.13	82.85	5.99
MST-AD	79.93	2.34	80.7	1.68	78.56	9.28

effectively reduce the ASR across all scenarios while maintaining robust model accuracy. Although the ASR remains relatively high under backdoor attacks, particularly for the MST-AD algorithm, it remains within acceptable limits given the extreme nature of this specific case.

VI. CONCLUSION

In this article, we proposed the FedDOT framework to allow the server to mitigate the impact of overwhelming targeted attackers. By leveraging model correlations and graph-theoretic concepts, FedDOT developed two strong attacker-detection algorithms, MST-AD and density-AD, and provided theoretical bounds to ensure the detection of adversaries. With an extensive set of experiments across different targeted attacks, including label flipping and backdoor attacks, we demonstrated the effectiveness of FedDOT in the presence of up to 70% of clients. By maintaining the lowest attack success rate and minimum false positives, both our algorithms are superior to existing defenses. In the future, we plan to scale up our framework to incorporate defenses against untargeted attacks. We will also explore robustness against compound attack scenarios that combine untargeted and targeted attacks in the FL framework.

REFERENCES

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.

[2] L. U. Khan, W. Saad, Z. Han, E. Hossain, and C. S. Hong, "Federated learning for internet of things: Recent advances, taxonomy, and open challenges," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1759–1799, Mar. 2021.

[3] R. S. Antunes, C. André da Costa, A. Küderle, I. A. Yari, and B. Eskofier, "Federated learning for healthcare: Systematic review and

architecture proposal," *ACM Trans. Intell. Syst. Technol. (TIST)*, vol. 13, no. 4, pp. 1–23, Apr. 2022.
[4] Z. Huang, F. Liu, and Y. Zou, "Federated learning for spoken language understanding," in *Proc. Int. Conf. Comput. Linguist.*, 2020, pp. 3467–3478.
[5] B. Nagy et al., "Privacy-preserving federated learning and its application to natural language processing," *Knowl.-Based Syst.*, vol. 23, no. 7, 2023, Art. no. 110475.
[6] C. Xie, K. Huang, P.-Y. Chen, and B. Li, "DBA: Distributed backdoor attacks against federated learning," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 145–157.
[7] C. Xie, M. Chen, P.-Y. Chen, and B. Li, "Crfl: Certifiably robust federated learning against backdoor attacks," in *Proc. Int. Conf. Mach. Learn.* PMLR, 2021, pp. 11372–11382.
[8] H. Wang et al., "Attack of the tails: Yes, you really can backdoor federated learning," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 16070–16084, 2020.
[9] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Proc. Eur. Symp. Res. Comput. Secur.* New York: Springer, 2020, pp. 480–501.
[10] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Int. Conf. Artif. Intell. Statist.* PMLR, 2020, pp. 2938–2948.
[11] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. Int. Conf. Mach. Learn.* PMLR, 2019, pp. 634–643.
[12] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *Proc. Int. Symp. Res. Attacks, Intrusions Defenses*, 2020, pp. 301–316.
[13] J. Steinhardt, P. W. Koh, and P. Liang, "Certified defenses for data poisoning attacks," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 3520–3532.
[14] A. Gupta, T. Luo, M. V. Ngo, and S. K. Das, "Long-short history of gradients is all you need: Detecting malicious and unreliable clients in federated learning," in *Proc. 27th Eur. Symp. on Res. Comput. Secur. (ESORICS 2022)*, Part III. New York: Springer, 2022, pp. 445–465.
[15] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 118–128.
[16] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5650–5659.
[17] S. Shen, S. Tople, and P. Saxena, "Auror: Defending against poisoning attacks in collaborative deep learning systems," in *Proc. 32nd Annu. Conf. Computer Secur. Appl.*, 2016, pp. 508–519.
[18] P. Ranjan, A. Gupta, F. Coro, Das, and Sajal, K., "Robust federated learning against backdoor attackers," in *Proc. IEEE INFOCOM -IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*. Piscataway, NJ, USA: IEEE Press, 2023, pp. 1–6.
[19] P. Ranjan, A. Gupta, F. Coro, and S. K. Das, "Securing federated learning against overwhelming collusive attackers," in *Proc. IEEE Global Commun. Conf. (Globecom)*, 2022, pp. 1448–1453.
[20] R. Al Mallah, D. Lopez, G. Badu-Marfo, and B. Farooq, "Untargeted poisoning attack detection in federated learning via behavior attestation," *IEEE Access*, vol. 34, pp. 471–486, 2023.
[21] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "Fltrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. 28th Annu. Netw. Distrib. System Secur. Symp. (NDSS)*, virtual, Feb. 21–25, The Internet Soc., 2021.
[22] L. Li, W. Xu, T. Chen, G. B. Giannakis, and Q. Ling, "RSA: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets," *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 1, pp. 1544–1551, 2019.
[23] X. Zheng, Q. Dong, and A. Fu, "WMDefense: Using watermark to defense byzantine attacks in federated learning," presented at the INFOCOM Workshop Secur. Privacy Big Data, 2022, pp. 1–6.
[24] Z. Zhang et al., "LSFL: A lightweight and secure federated learning scheme for edge computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, no. 3, pp. 365–379, Mar. 2022.
[25] Y. Chen, L. Su, and J. Xu, "Distributed statistical machine learning in adversarial settings: Byzantine gradient descent," *ACM Conf. Meas. Anal. Comput. Syst.*, vol. 1, no. 2, pp. 1–25, 2017.
[26] Z. Wu, Q. Ling, T. Chen, and G. B. Giannakis, "Federated variance-reduced stochastic gradient descent with robustness to byzantine attacks," *IEEE Trans. Signal Process.*, vol. 68, no. 5, pp. 4583–4596, May 2020.
[27] M. S. Ozdayi, M. Kantarcioglu, and Y. R. Gel, "Defending against backdoors in federated learning with robust learning rate," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 10, pp. 9268–9276, 2021.
[28] S. Li, Y. Cheng, W. Wang, Y. Liu, and T. Chen, "Learning to detect malicious clients for robust federated learning," 2020, *arXiv:2002.00211*.
[29] P. Ranjan, A. Gupta, and S. K. Das, "Securing federated learning from distributed backdoor attacks via maximal clique and dynamic reputation system," in *Proc. IEEE Int. Conf. Smart Comput. (SMARTCOMP)*. Piscataway, NJ, USA: IEEE Press, 2025, pp. 130–137.
[30] P. Ranjan, A. Gupta, and S. K. Das, "Robust federated learning against targeted attackers using model updates correlation," in *Handbook of Trustworthy Federated Learning*. New York: Springer, 2024, pp. 109–147.
[31] T. D. Nguyen et al., "{FLAME}: Taming backdoors in federated learning," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur. 22)*, 2022, pp. 1415–1432.
[32] P. Ranjan, F. Corò, A. Gupta, and S. K. Das, "Leveraging spanning tree to detect colluding attackers in federated learning," in *Proc. INFOCOM or IEEE Conf. Comput. Commun. Workshops*, 2022, pp. 1–2.
[33] Y. Mao, X. Yuan, X. Zhao, and S. Zhong, "Romoa: Robust model aggregation for the resistance of federated learning to model poisoning attacks," in *Proc. Eur. Symp. Res. Comput. Secur.* New York: Springer, 2021, pp. 476–496.
[34] S. Awan, B. Luo, and F. Li, "Contra: Defending against poisoning attacks in federated learning," in *Proc. Eur. Symp. Res. Comput. Secur.* New York: Springer, 2021, pp. 455–475.
[35] N. Wang, Y. Xiao, Y. Chen, Y. Hu, W. Lou, and Y. T. Hou, "Flare: Defending federated learning against model poisoning attacks via latent space representations," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2022, pp. 946–958.
[36] C. Xie, O. Koyejo, and I. Gupta, "Generalized byzantine-tolerant SGD," 2018, *arXiv:1802.10116*.
[37] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
[38] P. Ranjan, P. Khan, S. Kumar, and S. K. Das, "log-Sigmoid activation-based long short-term memory for time-series data classification," *IEEE Trans. Artif. Intell.*, vol. 5, no. 2, pp. 672–683, Feb. 2023.
[39] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proc. Amer. Math. Soc.*, vol. 7, no. 1, pp. 48–50, 1956.
[40] C. T. Dinh et al., "Federated learning over wireless networks: Convergence analysis and resource allocation," *IEEE/ACM Trans. Netw.*, vol. 29, no. 1, pp. 398–409, Jan. 2021.
[41] Y. Nesterov et al., *Lectures on Convex Optimization*, vol. 137. New York: Springer, 2018.
[42] Y. LeCun et al., "Backpropagation applied to handwritten zip code recognition," *Neural Comput.*, vol. 1, no. 4, pp. 541–551, 1989.
[43] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
[44] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," in *Proc. IEEE Int. Conf. Comput. Commun.*, Apr. 2009.
[45] X. Yan, L. Wu, Z. Zhang, B. Liu, L. Huo, and J. Wang, "Fedbap: Backdoor defense via benign adversarial perturbation in federated learning," in *Proc. 33rd ACM Int. Conf. Multimedia*, 2025, pp. 11947–11955.
[46] T. Clanuwat, M. Bober-Irizar, A. Kitamoto, A. Lamb, K. Yamamoto, and D. Ha, "Deep learning for classical Japanese literature," 2018, *arXiv:1812.01718*.

Priyesh Ranjan (Student Member, IEEE) received the B.Tech. degree in electrical engineering from the Department of Electrical Engineering, Indian Institute of Technology Patna, Patna, India, in 2017. He is currently working toward the Ph.D. degree in computer science with the Department of Computer Science, Missouri University of Science and Technology, Rolla, MO, USA.

His research interests include the field of machine learning, deep learning, cyber-physical systems, and smart healthcare.

Ashish Gupta (Member, IEEE) received the Ph.D. degree in computer science and engineering from the Indian Institute of Technology (BHU), Varanasi, India, in 2020.

He is an Assistant Professor in computer science with BITS Pilani, Dubai, UAE. He was working as a Postdoctoral Fellow with the Department of Computer Science, Missouri University of Science and Technology, Rolla, USA, from 2021 to 2023. His research interests include sensor data analytics, federated learning, and applied machine learning.

Federico Corò (Member, IEEE) received the M.Sc. degree from the University of Perugia, Perugia, Italy, in 2016, and the Ph.D. degree from Gran Sasso Science Institute, L'Aquila, Italy, in 2019, both in computer science.

From 2019 to 2022, he was a Postdoctoral Researcher in computer science with Sapienza University of Rome, Rome, Italy, and Missouri University of Science and Technology, Rolla, USA. He is currently an Assistant Professor with the University of Padua, Italy. His research interests include several aspects of theoretical computer science, such as combinatorial optimization, network analysis, and efficient implementation of algorithms.

Sajal K. Das (Fellow, IEEE) received the B.Tech. degree in 1983 from Calcutta University, the M.E. degree in 1985 from Indian Institute of Science, Bangalore, and the Ph.D. degree in 1988 from the University of Central Florida, Orlando, all in Computer Science. He is a Curator's Distinguished Professor of Computer Science and the Daniel St. Clair Endowed Chair with Missouri University of Science and Technology, Rolla, USA. His research interests include wireless and sensor networks, mobile and pervasive computing, mobile crowdsensing, cyber-physical systems and IoT, cloud and edge computing, cyber security, and social networks.

Prof. Das has coauthored more than 700 research articles in high-quality journals and refereed conference proceedings, five U.S. patents, and four books. His h-index is 99 with more than 42 000 citations. He serves as the Editor-in-Chief of *Pervasive and Mobile Computing Journal* (Elsevier's) and an Associate Editor of IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON MOBILE COMPUTING, and *ACM Transactions on Sensor Networks*.